

6.5830/1 Lecture 10

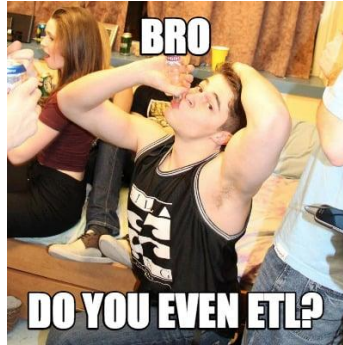


Column Stores

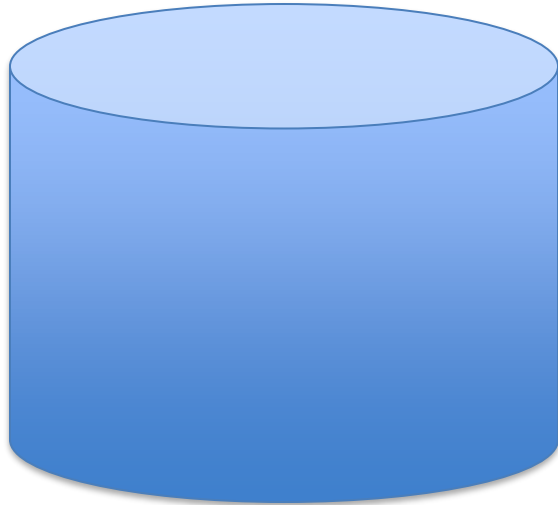
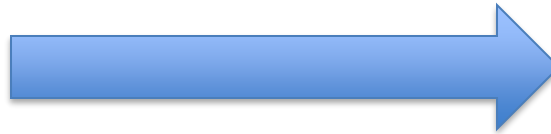
March 10, 2026

- Quiz 1 on March 12
- Lab 2 due March 19

Typical Database Setup



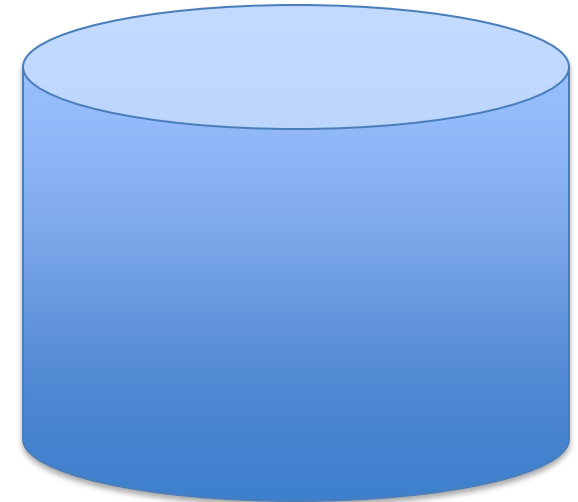
“Extract, Transform, Load”



Transactional database

Lots of writes/updates

Reads of individual records



Analytics / Reporting Database “Warehouse”

Lots of reads of many records

Bulk updates

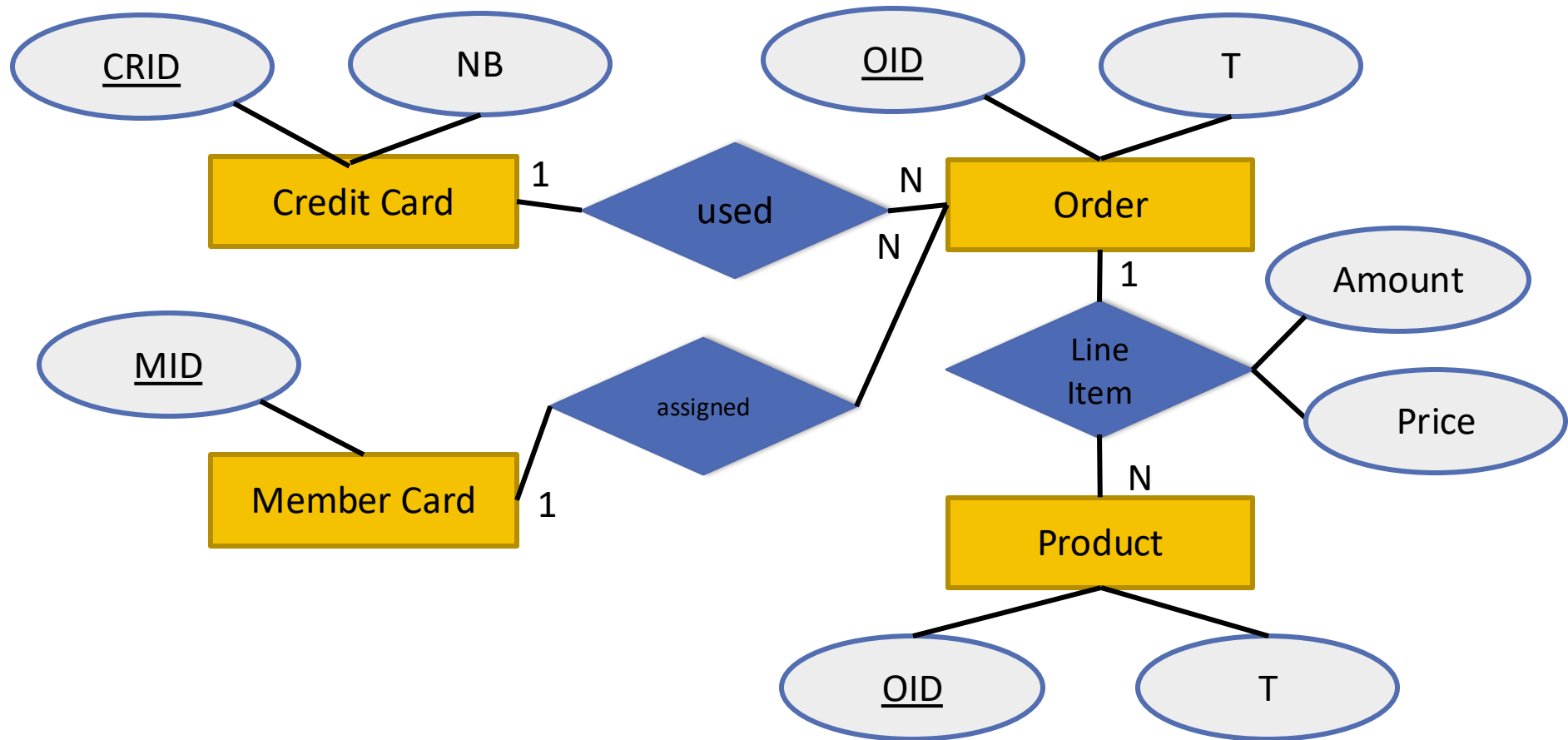
Typical query touches a few columns

PROBLEM

- **You are the new Data Scientist at New Market**
- New Market is tracking all customer purchases with their membership card or credit card
- They also have data about their customers (estimated income, family status,...)
- Recently, they are trying to improve their image for young parents
- As a start they want to know the following information for parents under 30 for 2026:
 - How much do they spend?
 - How much do they spend per state?
 - How does this compare to all customers under 30?
 - What are their favorite products?
 - How much do they spend per year?

Your first project: Design the schema for New Market!

TYPICAL OLTP SCHEMA



OLAP Star Schema

Shop
- <u>Shop ID</u>
- Business_Type
- City
- City_Population
- State
- ...

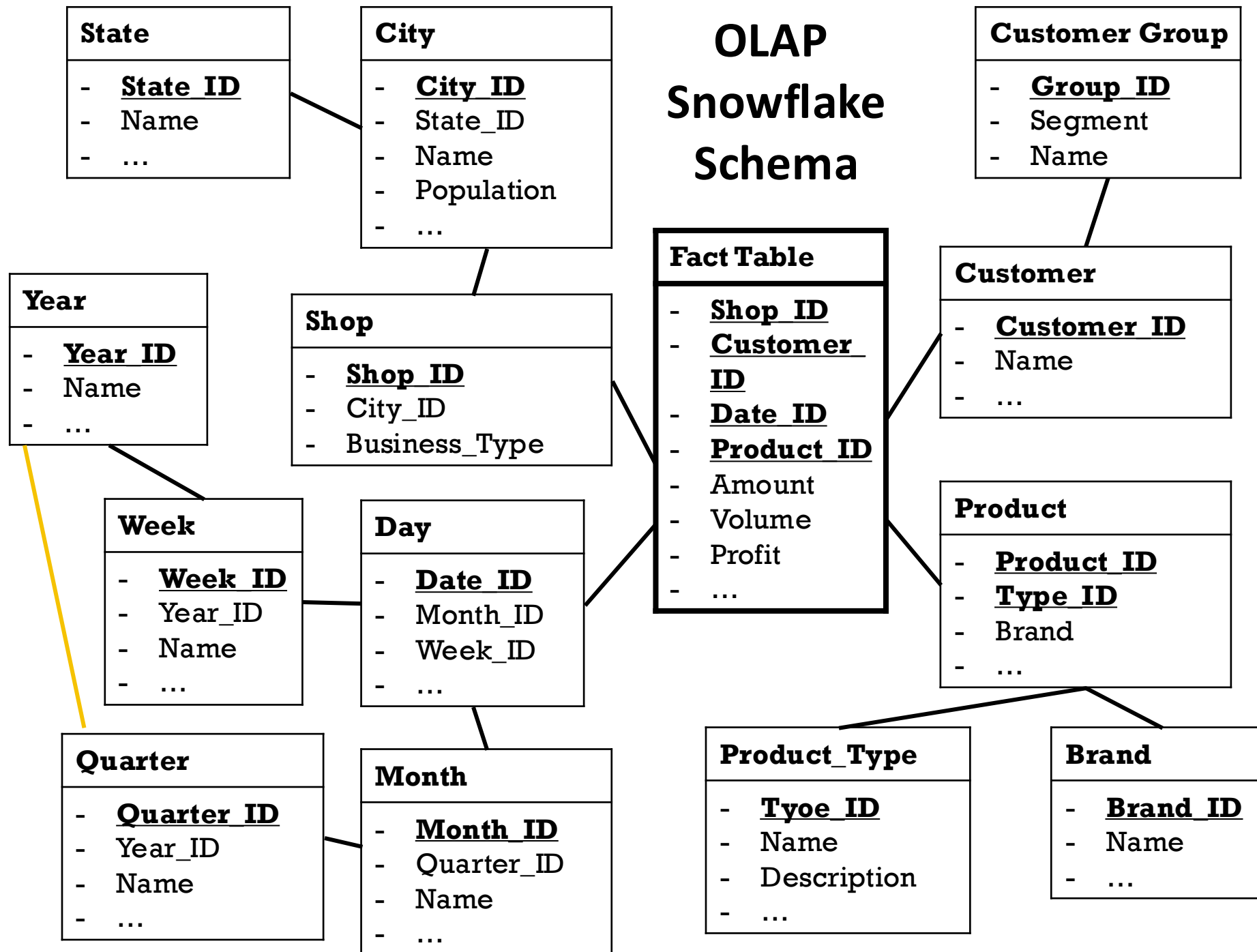
Customer
- <u>Customer ID</u>
- Name
- Segment
- Group_Name
- ...

Fact Table
- <u>Shop ID</u>
- <u>Customer ID</u>
- <u>Date ID</u>
- <u>Product ID</u>
- Amount
- Volume
- Profit
- ...

Time
- <u>Date ID</u>
- Month
- Quarter
- Year
- ...

Product
- <u>Product ID</u>
- <u>Type</u>
- Brand
- Description
- ...

OLAP Snowflake Schema

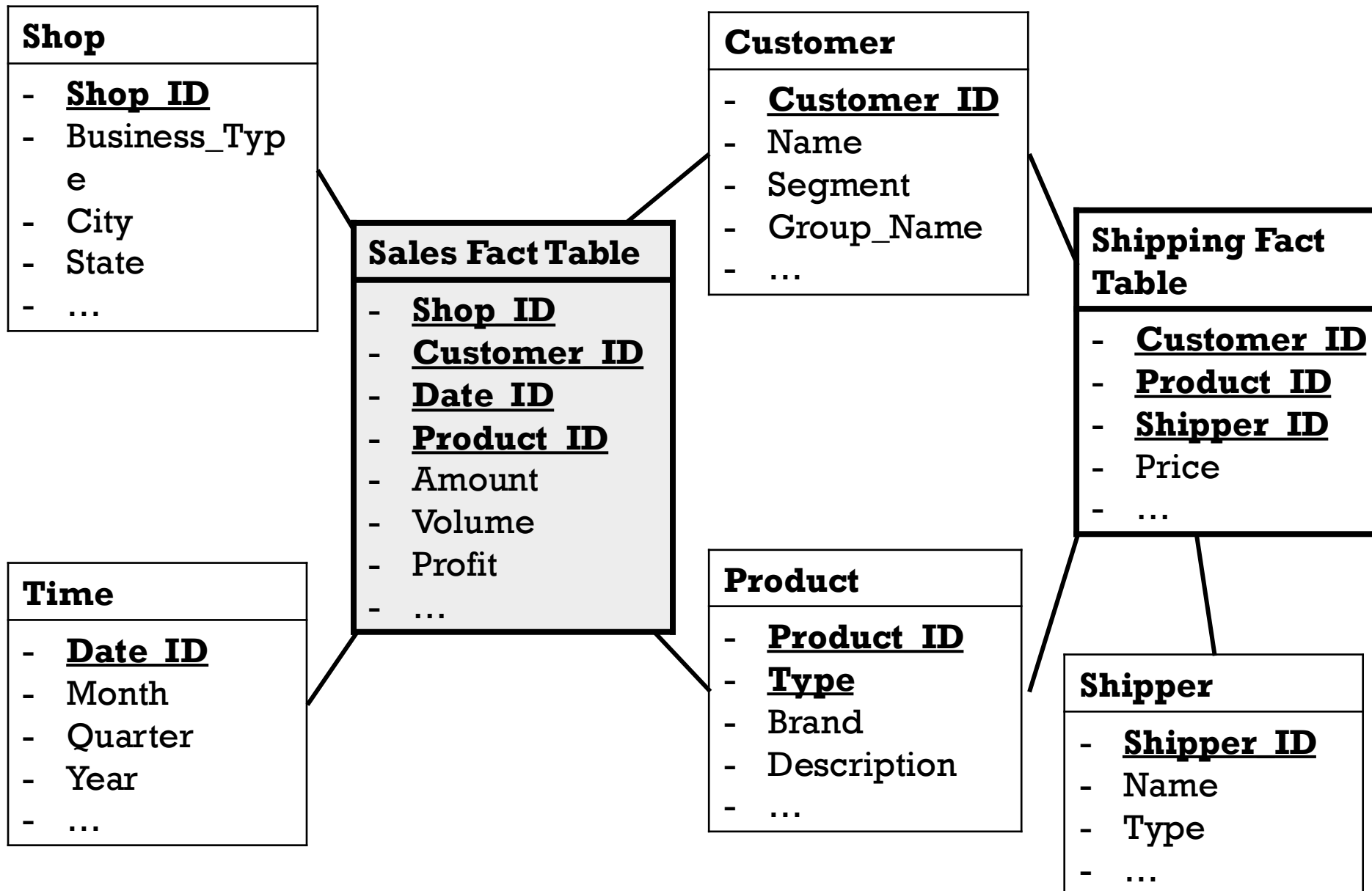


STAR VS. SNOWFLAKE SCHEMA

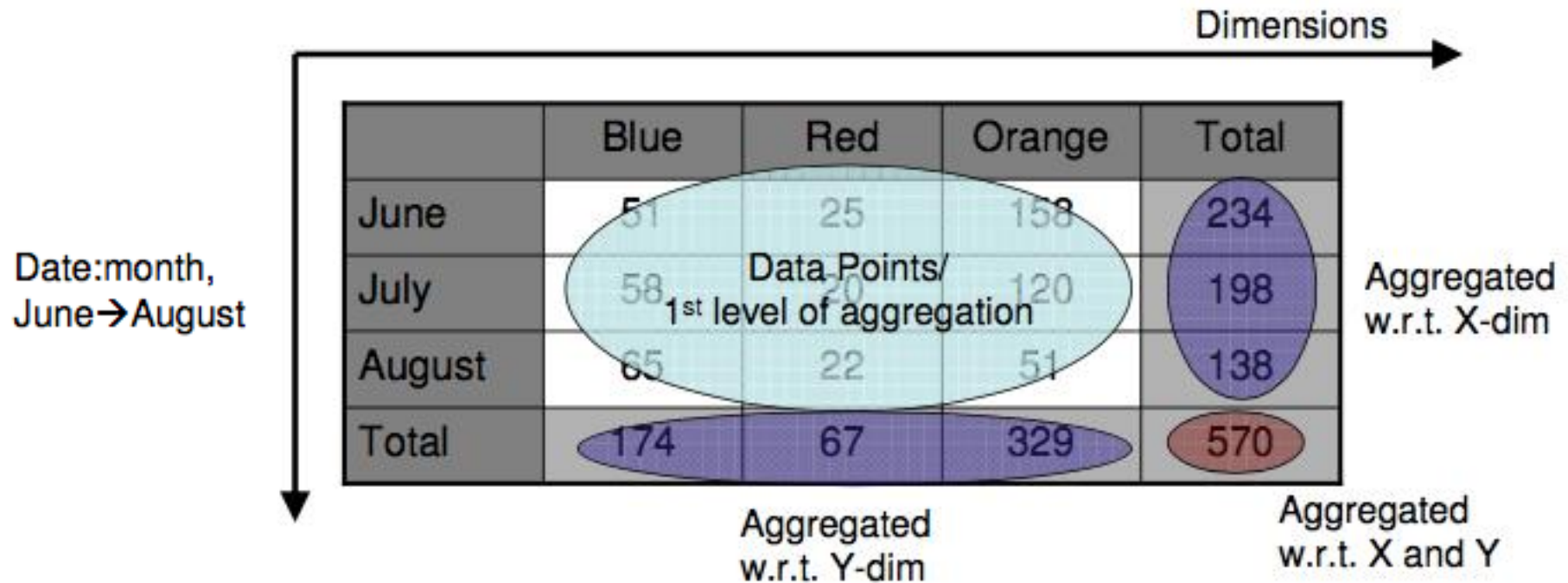
	Snowflake	Star
Normalization/De-Normalization	Dimension Tables are in Normalized form but Fact Table is still in De-Normalized form	Both Dimension and Fact Tables are in De-Normalized form
Space	Smaller	Bigger (Redundancy)
Query Performance	More Joins → slower	Fewer Joins → faster
Ease of Use	Complex Queries	Pretty Simply Queries
When to use	When dimension table is relatively big in size, snowflaking is better as it reduces space.	When dimension table contains less number of rows, we can go for Star schema.

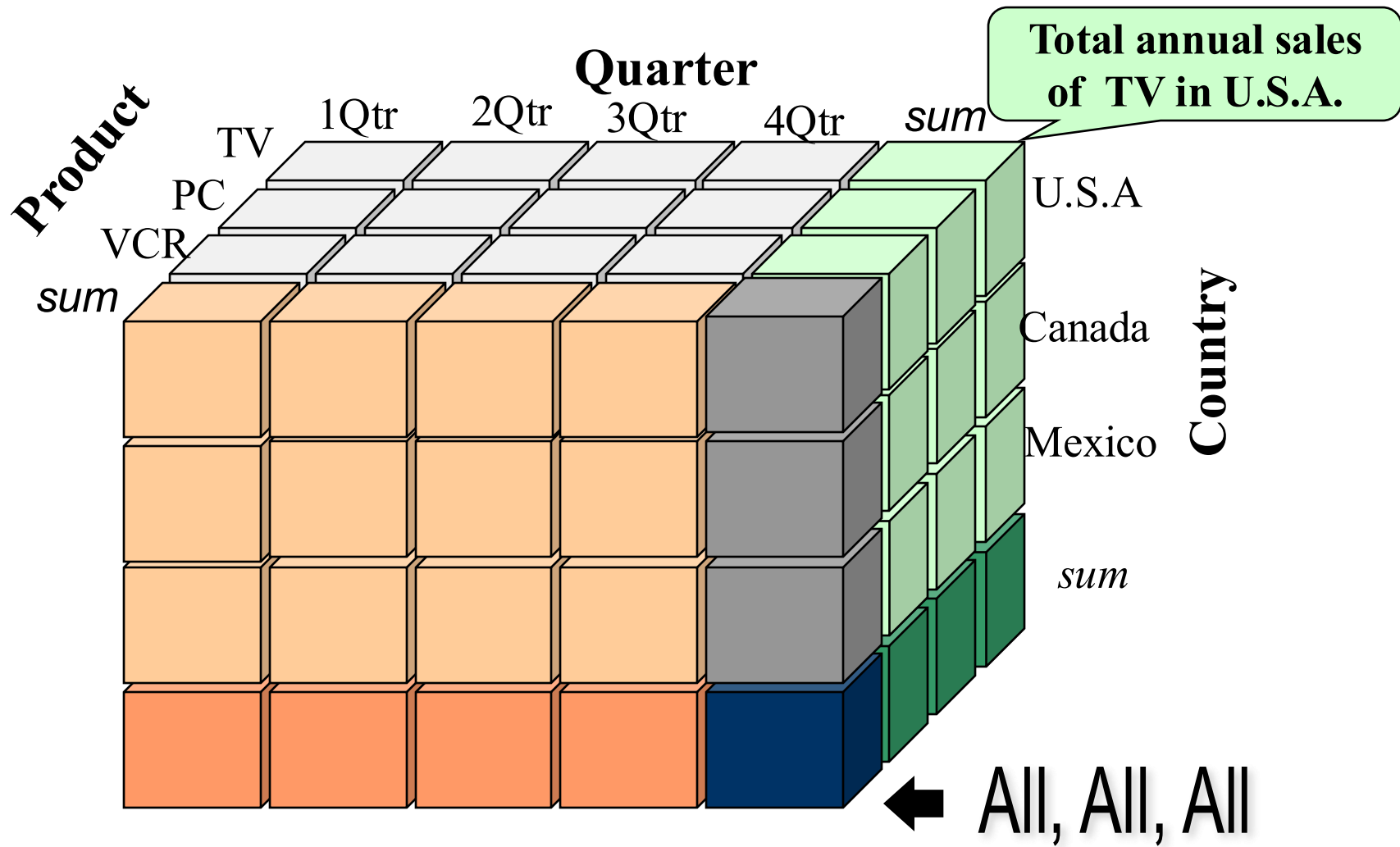
Galaxy / Fact Constellation

Schema



2 DIMENSIONAL CASE





TYPICAL OLAP OPERATIONS

Roll up (drill-up): summarize data

by climbing up hierarchy or by dimension reduction

Drill down (roll down): reverse of roll-up

from higher level summary to lower level summary or detailed data, or introducing new dimensions

Slice and dice: project and select

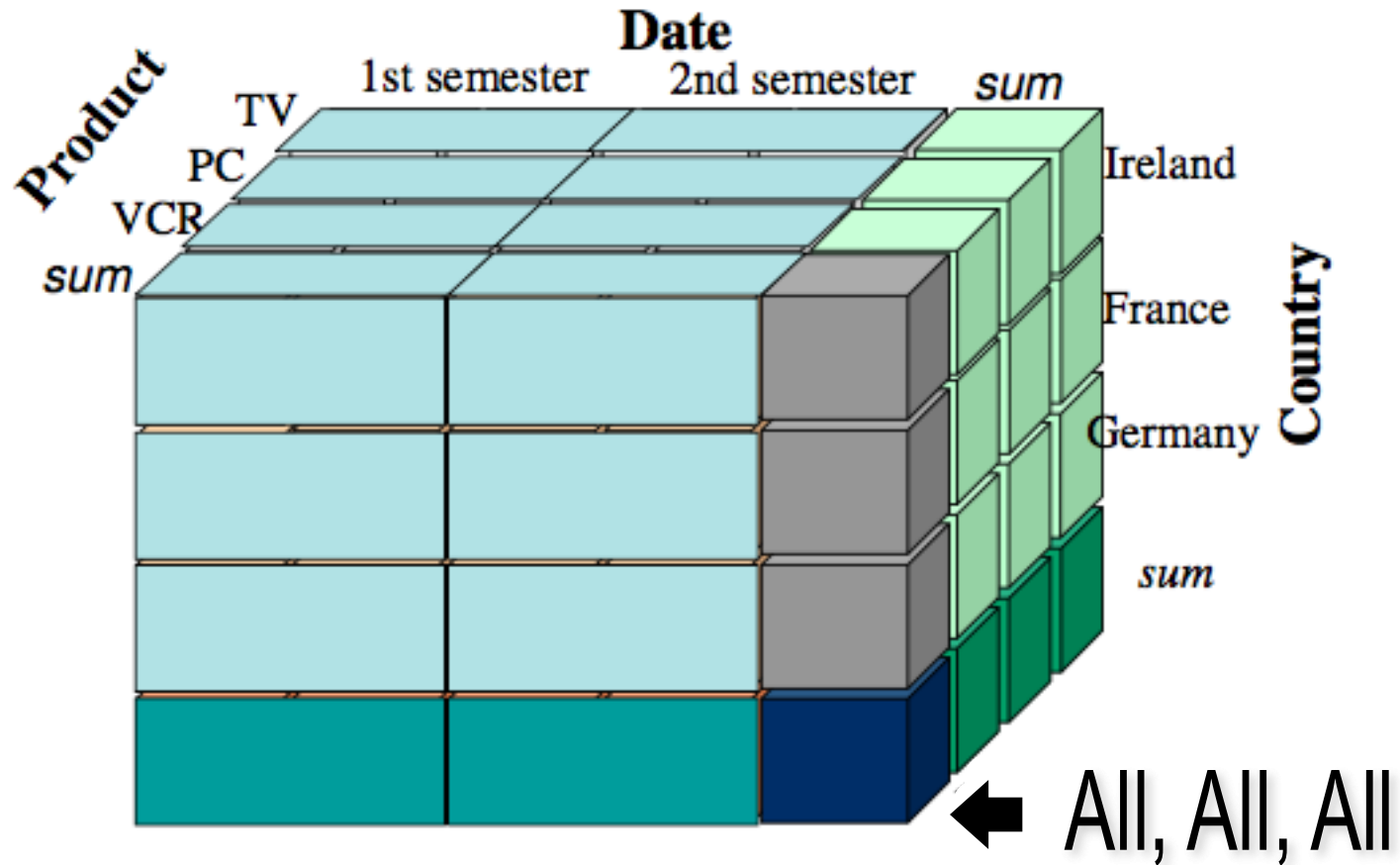
Pivot (rotate): reorient the cube, visualization, 3D to series of 2D planes.

Other operations

drill across: involving (across) more than one fact table

drill through: through the bottom level of the cube to its back-end relational tables (using SQL)

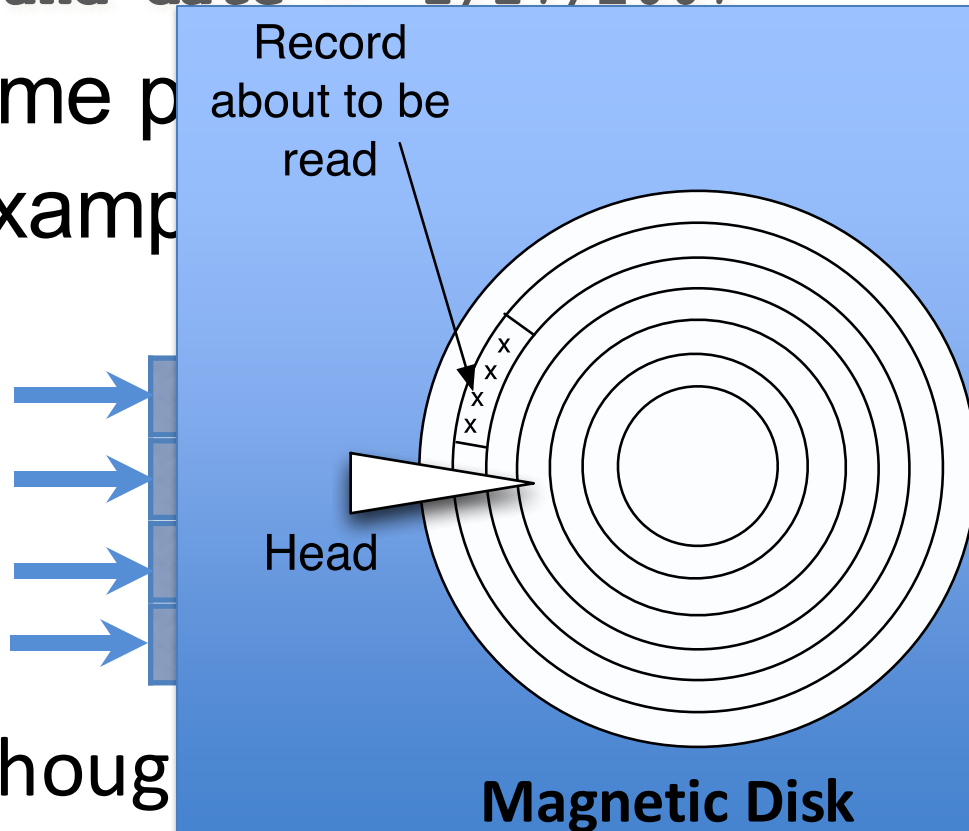
ROLLUP



How Long Does a Scan Take?

```
SELECT avg(price) FROM tickstore WHERE symbol =  
'GM' and date = '1/17/2007'
```

- Time to read data
- Example



change	date
YSE	1/17/2007
YSE	1/17/2007
YSE	1/17/2007
QDS	1/17/2007

Even though
if data is on
Memory and SSD also transfer a block at a time, so
same issue arises.

Column Representation Reduces Scan Time

- Idea: Store each column in a separate file

Column Representation

Reads Just 3 Columns

GM	30.77	1,000	NYSE	1/17/2007
GM	30.77	10,000	NYSE	1/17/2007
GM	30.78	12,500	NYSE	1/17/2007
AAPL	93.24	9,000	NQDS	1/17/2007

Assuming each column is same size, reduces bytes read from disk by factor of 3/5

In reality, databases are often 100's of columns

Linearizing a Table – Row store

C1	C2	C3	C4	C5	C6

Memory/Disk
(Linear Array)

R1 C1
R1 C2
R1 C3
R1 C4
R1 C5
R1 C6
R2 C1
R2 C2
R2 C3
R2 C4
R2 C5
R2 C6
R3 C1
R3 C2
R3 C3
R3 C4
R3 C5
R3 C6
R4 C1
R4 C2
R4 C3
R4 C4
R4 C5
R4 C6

Linearizing a Table – Column Store

C1	C2	C3	C4	C5	C6

Memory/Disk
(Linear Array)

R1 C1
R2 C1
R3 C1
R4 C1
R5 C1
R6 C1
R1 C2
R2 C2
R3 C2
R4 C2
R5 C2
R6 C2
R1 C3
R2 C3
R3 C3
R4 C3
R5 C3
R6 C3
R1 C4
R2 C4
R3 C4
R4 C4
R5 C4
R6 C4

Tables Often Super Wide

- Data warehouse at Cambridge Mobile Telematics

Table	#columns
t1	251
t2	248
t3	134
t4	107
t5	87
t6	83
t7	71
t8	54
t9	52
t10	45

Average query access 4-5 fields

Top 2-3 tables involved in nearly every query

Using a row-store would impose $\sim 200/4 = 50\times$ performance overhead

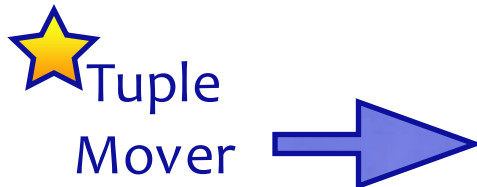
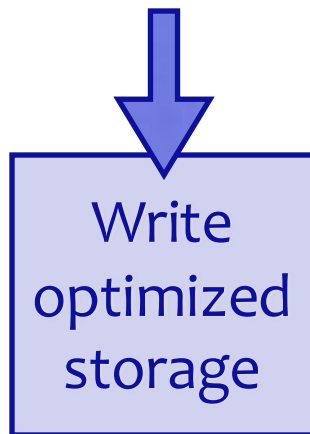
When Are Columns Right?

- **Warehousing (OLAP)**
 - Read-mostly; batch update
 - Queries: Scan and aggregate a few columns
- **Vs. Transaction Processing (OLTP)**
 - Write-intensive, mostly single record ops.
- **Column-stores: OLAP optimized**
- **In practice >10x performance on comparable HW, for many real world analytic applications**
 - True even if w/ Flash or main memory!

Different architectures for different workloads

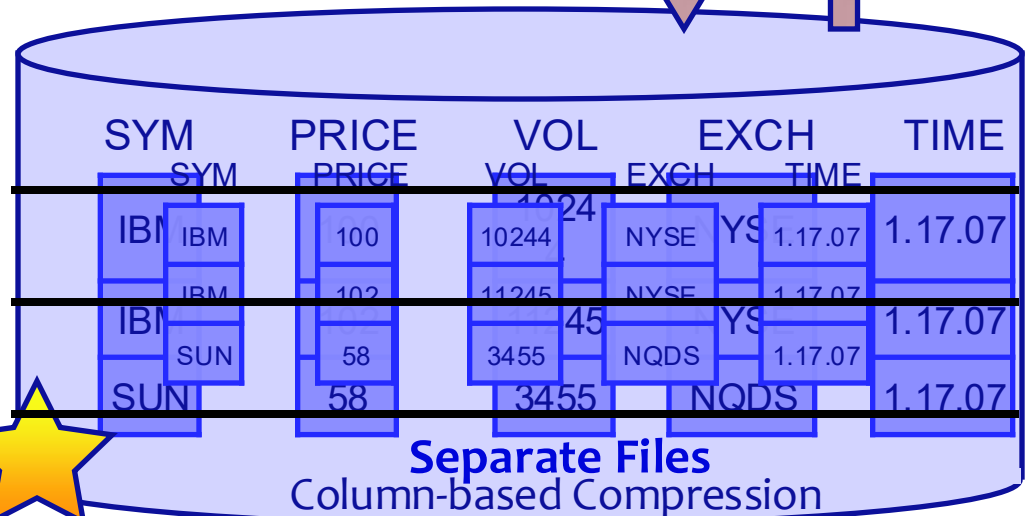
C-Store: Rethinking Database Design from the Ground Up

Inserts



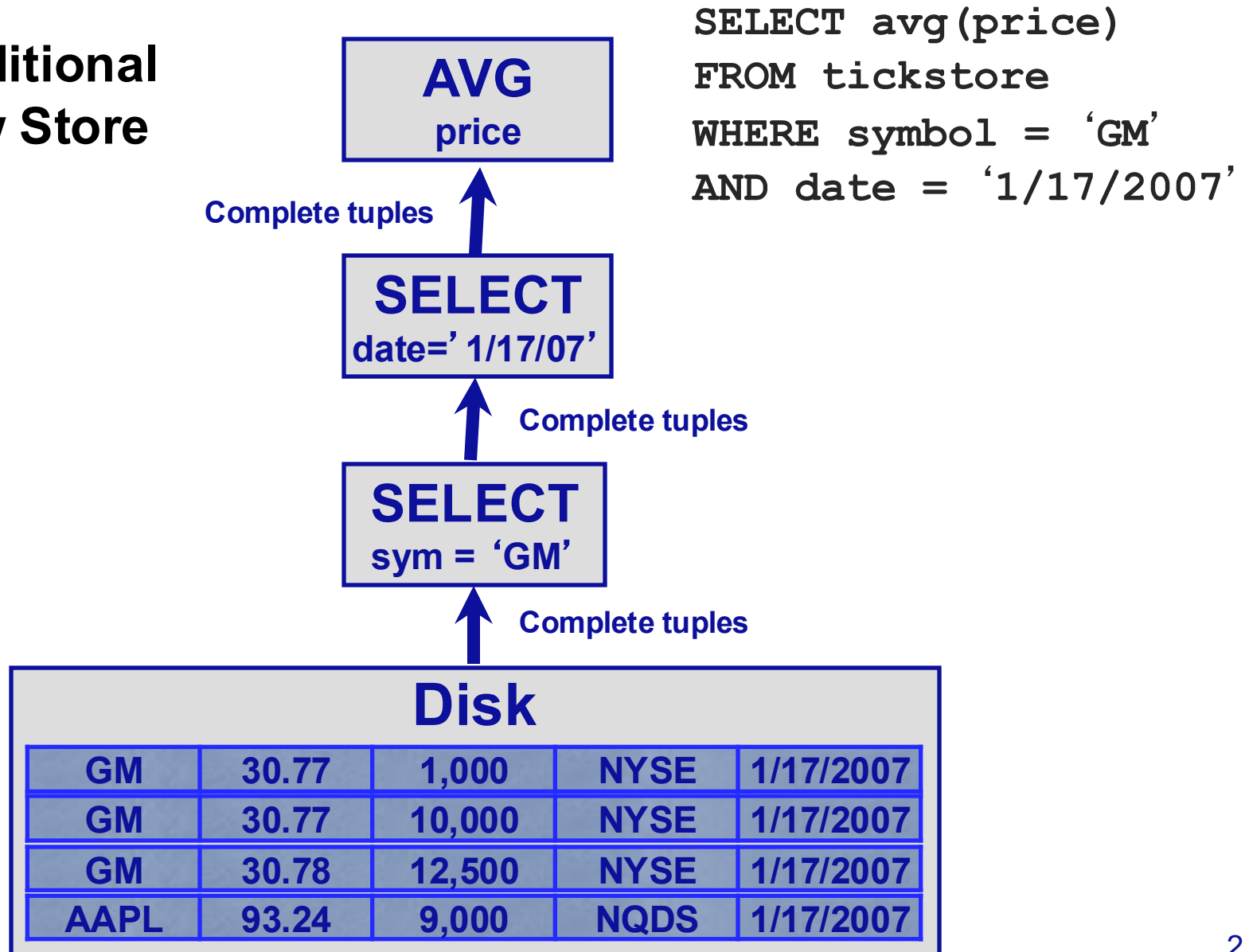
Shared nothing horizontal partitioning

Column-oriented query executor



Query Processing Example

- Traditional Row Store

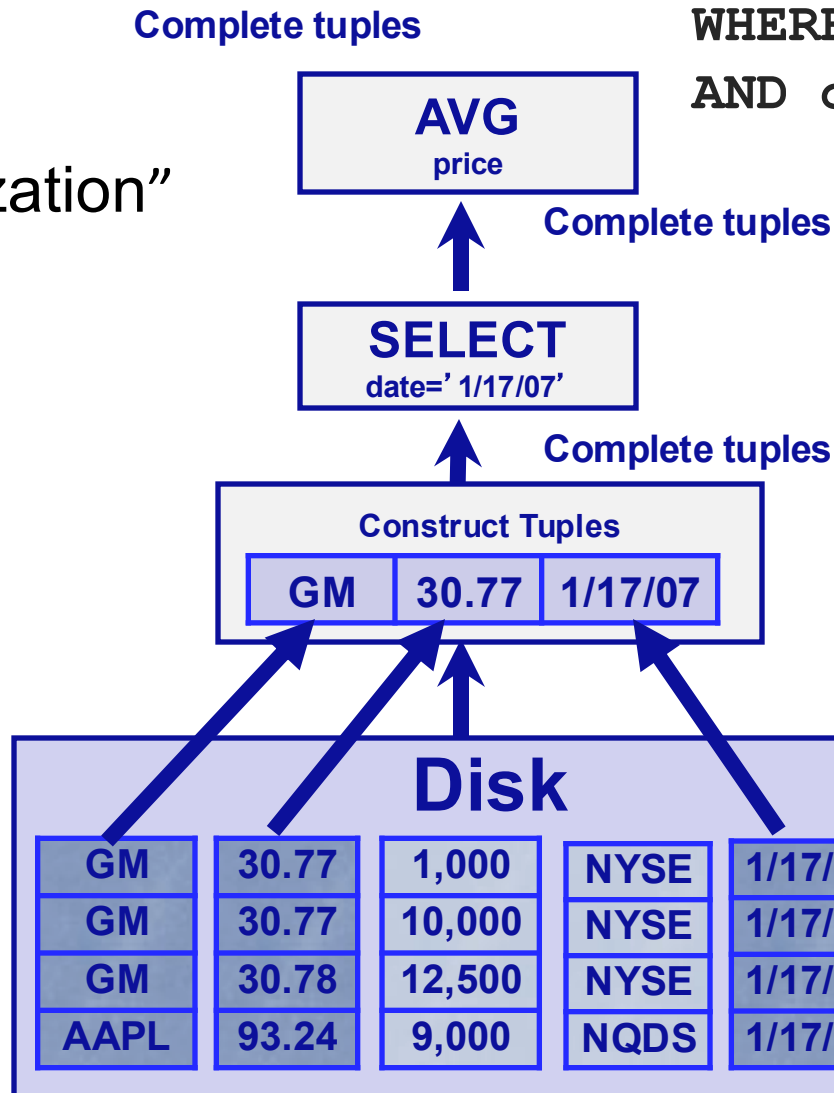


Query Processing Example

- **Basic Column Store**

- “Early Materialization”

```
SELECT avg(price)
FROM tickstore
WHERE symbol = 'GM'
AND date = '1/17/2007'
```

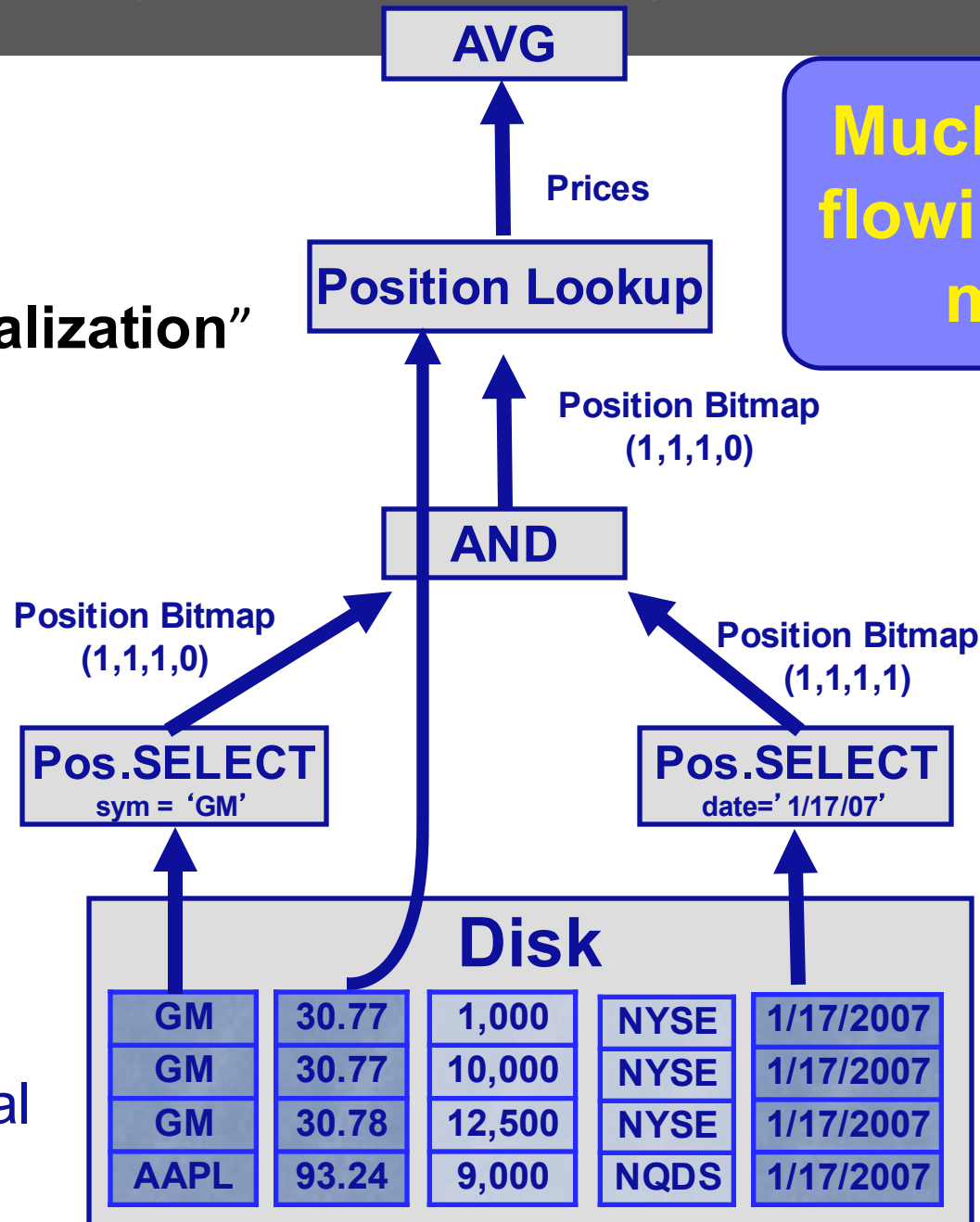


Row-oriented plan

Fields from same tuple at same index (position) in each column file

Query Processing Example

- C-Store
- “Late Materialization”



**Much less data
flowing through
memory**

See Abadi et al
ICDE 07

Why Compress?

- **Database** size is 2x-5x larger than the volume of data loaded into it
- Database performance is proportional to the amount of data flowing through the system

Column-Oriented Compression

- Query engine processes compressed data
- Transfers load from storage to CPU
- Multiple compression types
 - Run-Length Encoding (RLE), LZ, Delta Value, Block Dictionary Bitmaps, Null Suppression
- System chooses which to apply
- Typically see 50% - 90% compression
- NULLs take virtually no space

Columns contain similar data, which makes compression easy

RLE	Delta	LZ	RLE	RLE
3xGM	30.77	1,000	3xNYSE	4 x 1/17/2007
1XAPPL	30.77	10,000	1XNQDS	1/17/2007
GM	30.78	12,500	NYSE	1/17/2007
AAPL	93.24	9,000	NQDS	1/17/2007

Run Length Encoding

- **Replace repeated values with a count and a value**
- **For single values, use a run length of 1**
 - **Naively, can increase storage space**
 - **Can use a shorter bit sequence for 1s, at the cost of more expensive decompression**
- **E.g., 1110002 $\rightarrow$ 3x1, 3x0, 1x2**
- **Works well for mostly-sorted, few-valued columns**

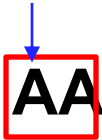
Dictionary Encoding

- **Many variants; simplest is to replace string values with integers and maintain a dictionary**
- **I.e., AAPL, AAPL, IBM, MSFT →
1,1,2,3 + 1:AAPL, 2:IBM, 3:MSFT**
- **Works well for few-valued string columns**
 - **Choice of dictionary not obvious**
 - **Words? Records?**

Lempel Ziv Encoding

- LZ (“Lempel Ziv”) Compression
- General purpose lossless data compression
- Builds data dictionary dynamically as it runs
 - Add new bit strings to the dictionary as they are encountered
- Treat entire column as a document
- There are *many* LZ variants

LZ Example

-  AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16

Output:

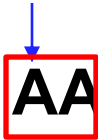
LZ Example

-  **AAPLAAPLIBMAAPL**

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16

Output: 1

LZ Example

-  **AAPLAAPLIBMAAPL**

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, .., AA:27

Output: 1

LZ Example

-  **AAPLAAPLIBMAAPL**

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, .., AA:27,
AP:28

Output: 1 1

LZ Example

-  **AAPLAAPLIBMAAPL**

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, .., AA:27,
AP:28

Output: 1 1 16

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, .., AA:27,
AP:28, PL: 29

Output: 1 1 16 12

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, ..., AA:27,
AP:28, PL: 29, LA: 30

Output: 1 1 16 12

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, ..., AA:27,
AP:28, PL: 29, LA: 30

Output: 1 1 16 12 27

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, ..., AA:27,
AP:28, PL: 29, LA: 30, AAP:31

Output: 1 1 16 12 27

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, .., AA:27,
AP:28, PL: 29, LA: 30, AAP:31

Output: 1 1 16 12 27

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, .., AA:27,
AP:28, PL: 29, LA: 30, AAP:31, PLI: 32

Output: 1 1 16 12 27 29

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, ..., AA:27,
AP:28, PL: 29, LA: 30, AAP:31, PLI: 32, LI: 33, IB: 34

Output: 1 1 16 12 27 29 9

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, ..., AA:27,
AP:28, PL: 29, LA: 30, AAP:31, PLI: 32, LI: 33, IB: 34, BM: 35

Output: 1 1 16 12 27 29 9 2

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, .., AA:27,
AP:28, PL: 29, LA: 30, AAP:31, PLI: 32, LI: 33, IB: 34, BM: 35, MA:36

Output: 1 1 16 12 27 29 9 2 13

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, .., AA:27,
AP:28, PL: 29, LA: 30, AAP:31, PLI: 32, LI: 33, IB: 34, BM: 35, MA:36

Output: 1 1 16 12 27 29 9 2 13

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, .., AA:27,
AP:28, PL: 29, LA: 30, AAP:31, PLI: 32, LI: 33, IB: 34, BM: 35, MA:36

Output: 1 1 16 12 27 29 9 2 13 31

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, .., AA:27,
AP:28, PL: 29, LA: 30, AAP:31, PLI: 32, LI: 33, IB: 34, BM: 35, MA:36
AAPL:37

Output: 1 1 16 12 27 29 9 2 13 31

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, .., AA:27,
AP:28, PL: 29, LA: 30, AAP:31, PLI: 32, LI: 33, IB: 34, BM: 35, MA:36
AAPL:37

Output: 1 1 16 12 27 29 9 2 13 31 12

LZ Example

- AAPLAAPLIBMAAPL

Dictionary: A:1, B:2, ... ,F:6, ... ,I:9, ..., L:12, M:13, ..., P:16, .., AA:27,
AP:28, PL: 29, LA: 30, AAP:31, PLI: 32, LI: 33, IB: 34, BM: 35, MA:36
AAPL:37

Output: 1 1 16 12 27 29 9 2 13 31 12

Reduced from 15 to 11 symbols

But future AAPL patterns will be emitted as 1 byte instead of 4

Dictionary can be further encoded, e.g., using entropy encoding to make most common patterns use least bits (“Huffman encoding”)

Bit Packing

- **Encode values with fewest possible bits, not the standard fixed size**
- **Each value becomes bit-length (e.g., 0-8 or 0-32), followed by value in that many bits**
- **E.g.,: 1, 2, 37, 7**
 - **Need 1, 2, 6, and 3 bits respectively**
 - **Each number has 3 bit header and encoded value**
 - 1: 0x001, 0x1
 - 2: 0x010, 0x10
 - 37: 0x110, 0x100101
 - 7: 0x011, 0x111

• **$3 \times 4 + 12 = 24$ bits to encode vs e.g. $8 \times 4 = 32$**

Delta Encoding

- **Consecutive values encoding as difference to previous values**
- 1.1, 1.2, 1.3 $\rightarrow$ 1.1, +.1, +1
 - After encoding as deltas, bit-pack
 - Works if deltas can be represented in fewer bits than whole values
- Works well for e.g., floats with small variations

Bitmap Encoding

- Encode few-valued columns as bitmaps
- Each unique value gets a corresp bitmap vector; it describes the rows where the value exists
- M M M F F $\rightarrow$ M:11100, F:00011
 - If fewer distinct values than bitwidth of field, saves space
 - Bitmaps can be further compressed, e.g., using RLE
- Bitmaps are very good for certain kinds of operations, e.g., filtering

Sorted Data

- Delta and RLE work great on sorted data
- Trick: Secondary sorting

X	Y
a	2
b	2
a	1
b	1

Sort on X,
then Y

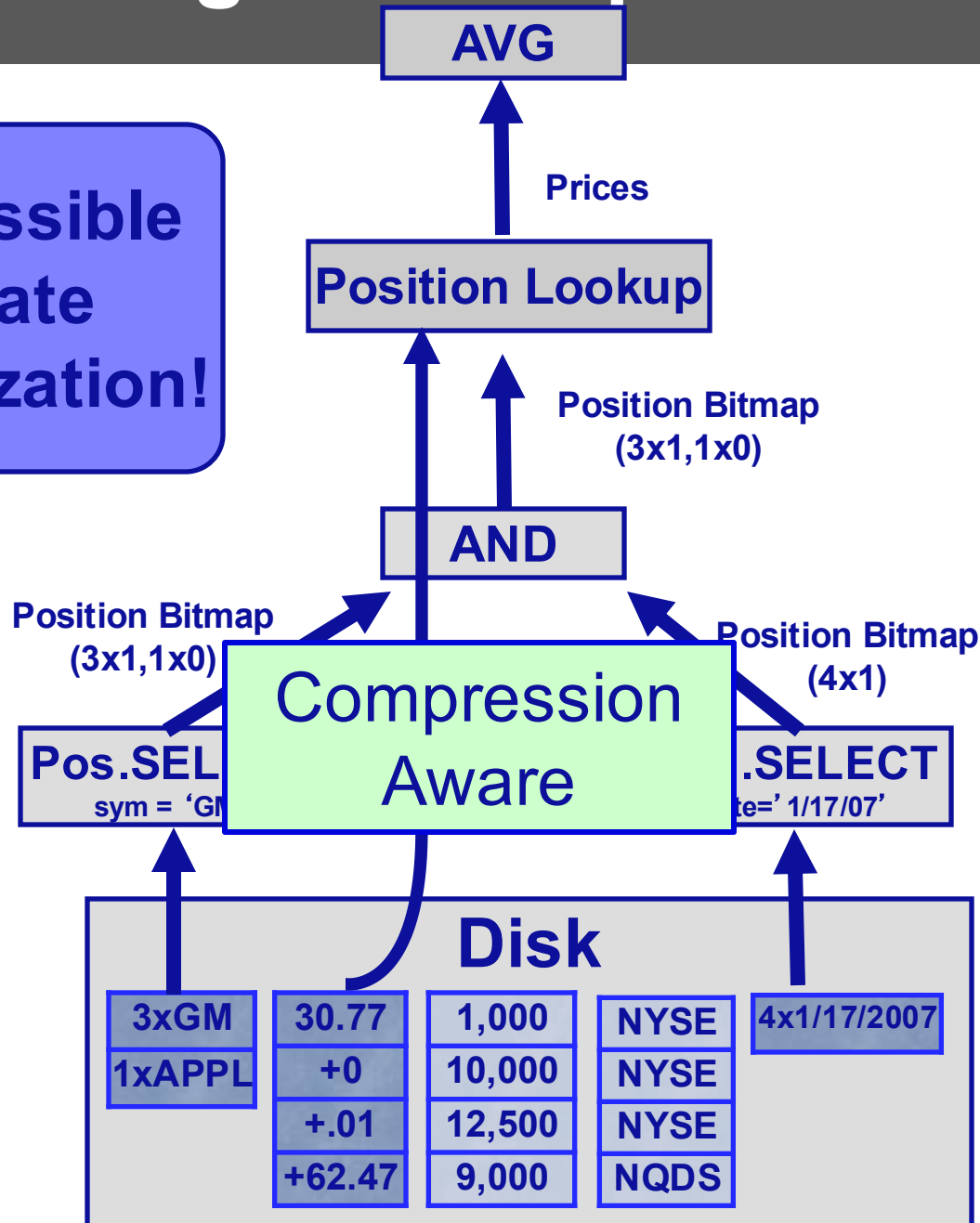


X	Y
a	1
a	2
b	1
b	2

Y is not
sorted,
but if
many
duplicates
of X, will
be
“mostly”
sorted

Operating on Compressed Data

Only possible
with late
materialization!



Direct Operation Optimizations

- **Compressed data used directly for position lookup**
 - **RLE, Dictionary, Bitmap**
- **Direct Aggregation and GROUP BY on compressed blocks**
 - **RLE, Dictionary**
- **Join runs of compressed blocks**
 - **RLE, Dictionary**
- **Min/max directly extracted from sorted data**

Compression + Sorting is a Huge Win

- **How can we get more sorted data?**
- **Store duplicate copies of data**
 - **Use different physical orderings**
- **Improves ad-hoc query performance**
 - **Due to ability to directly operate on sorted, compressed data**
- **Supports fail-over / redundancy**

Study Break: Compression

For each of the following columns, what compression method would you recommend?

(Choose from A. RLE, B. Dictionary, C. Bitmap, D. Delta, E. Bit-packing)

<https://clicker.mit.edu/6.5830/>

An unsorted column of integers in the range 0-100
Delta/Bit-packing (LZ/dictionary also OK)

A mostly sorted LZ column of arbitrary strings

A mostly sorted RLE column of integers in the range 0-10
Delta

A sorted column of floats
Bitmap

Write Performance

Trickle load: Very
Fast Inserts

> **Write-optimized
Column Store
(WOS)**

Memory: mirrored
projections in
insertion order
(uncompressed)

**Queries read
from both WOS
and ROS**

Tuple Mover

Asynchronous Data
Movement

Batched

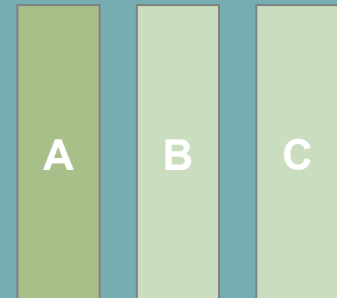
Amortizes seeks

**Amortizes
recompression**

**Enables continuous
load**

> **Read-optimized
Column Store (ROS)**

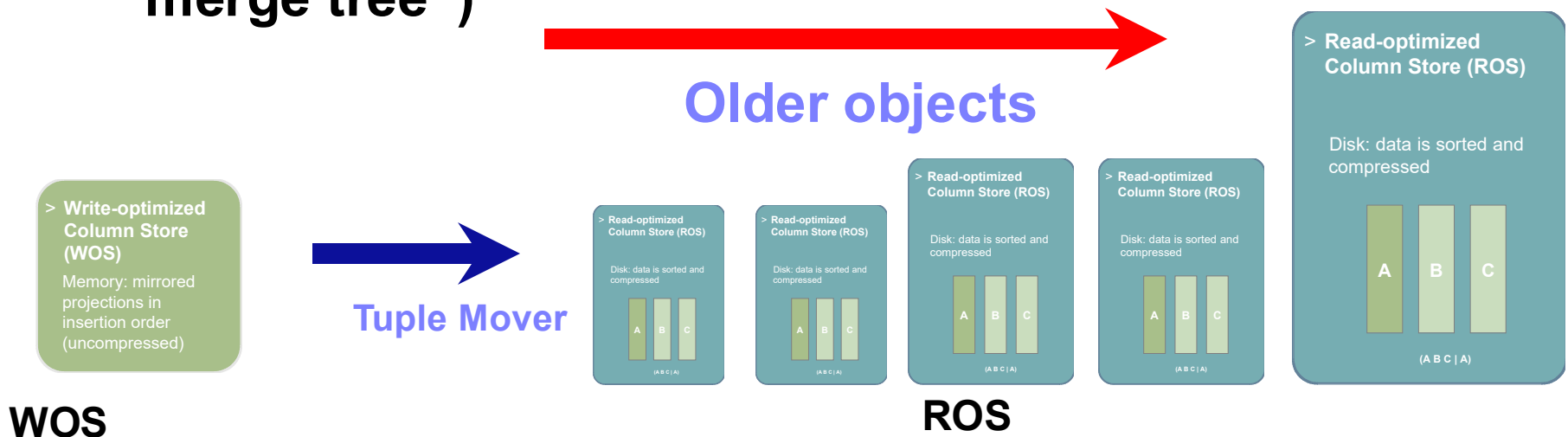
Disk: data is sorted and
compressed



(A B C | A)

When to Rewrite ROS Objects?

- Store multiple ROS objects, instead of just one
 - Each of which must be scanned to answer a query
- Tuple mover writes new objects
 - Avoids rewriting whole ROS on merge
- Periodically merge ROS objects to limit number of distinct objects that must be scanned (“Log structured merge tree”)



Problem: Lots of Partitions

- Performance will degrade as you get many partitions
- Idea: merge some partitions together, but how?
- Log structured merge tree: arrange so partitions merge a logarithmic number of times



Problem: Lots of Partitions

- Performance will degrade as you get many partitions
- Idea: merge some partitions together, but how?
- Log structured merge tree: arrange so partitions merge a logarithmic number of times



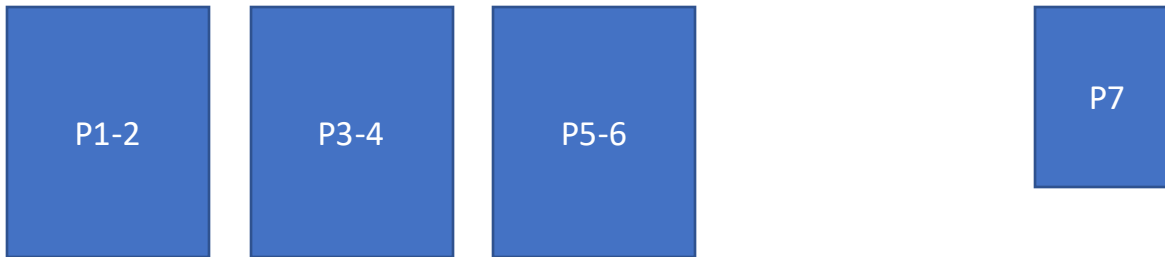
Problem: Lots of Partitions

- Performance will degrade as you get many partitions
- Idea: merge some partitions together, but how?
- Log structured merge tree: arrange so partitions merge a logarithmic number of times



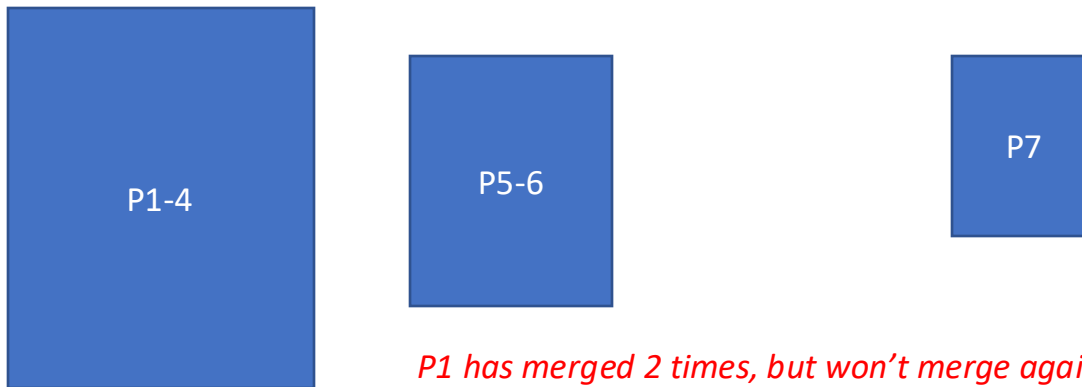
Problem: Lots of Partitions

- Performance will degrade as you get many partitions
- Idea: merge some partitions together, but how?
- Log structured merge tree: arrange so partitions merge a logarithmic number of times



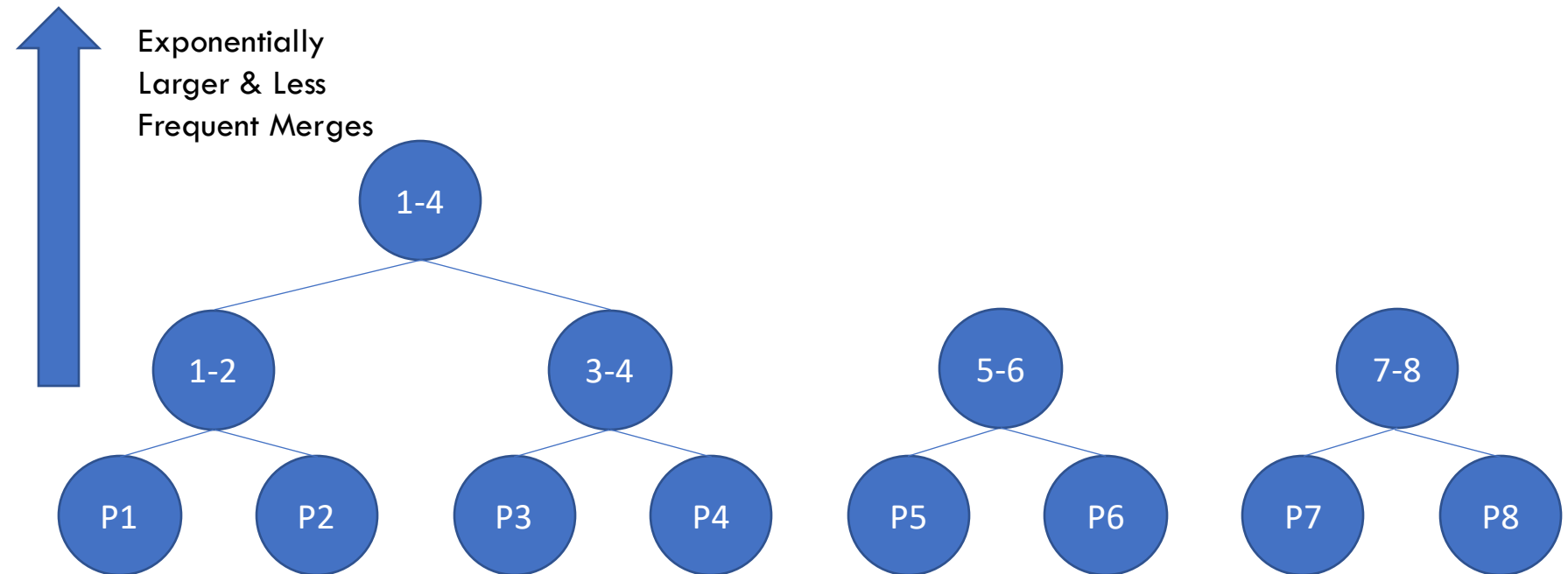
Problem: Lots of Partitions

- Performance will degrade as you get many partitions
- Idea: merge some partitions together, but how?
- Log structured merge tree: arrange so partitions merge a logarithmic number of times



P1 has merged 2 times, but won't merge again until after 8 more partitions arrive

Log Structure Merge Tree



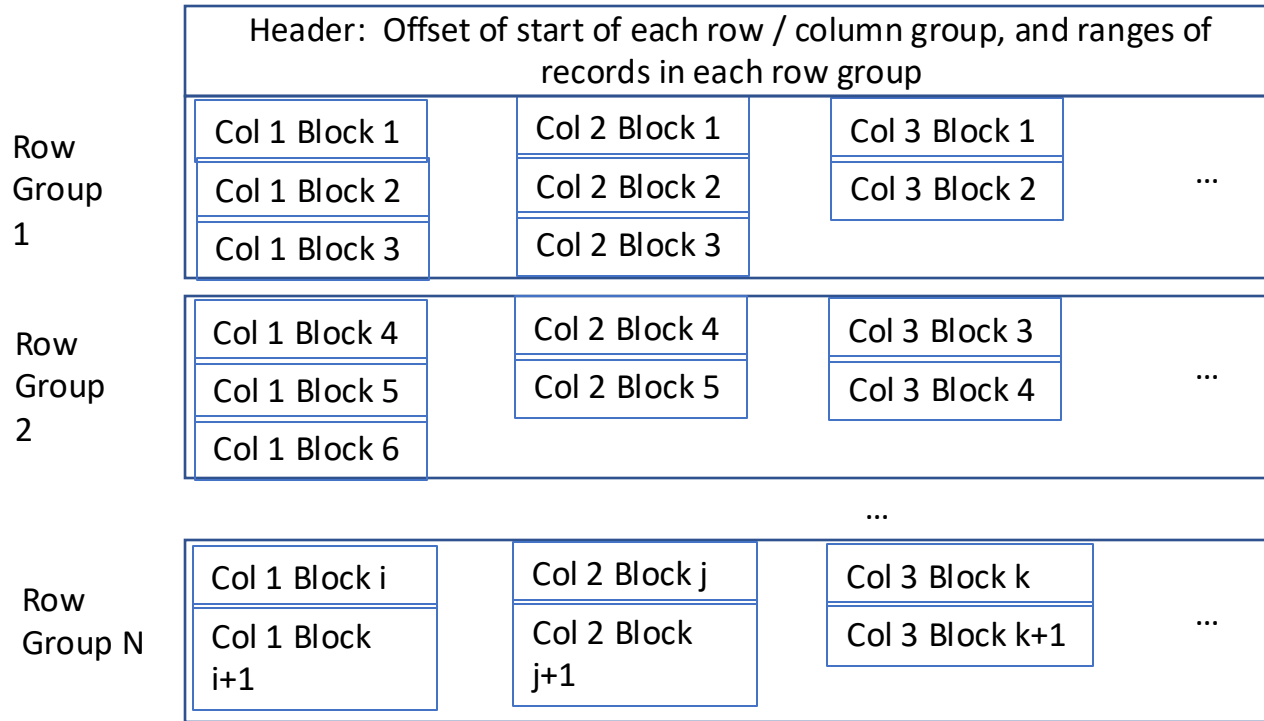
Efficient Data Loading: Parquet

- Parquet is column-oriented file format that is MUCH more efficient than CSV for storing tabular data
- Vs CSV, Parquet is stored in binary representation
 - Uses less space
 - Doesn't require conversion from strings to internal types
 - Doesn't require parsing or error detection
 - Column-oriented, making access to subsets of columns much faster



Parquet Format

- Data is partitioned sets of rows, called “row groups”
- Within each row group, data from different columns is stored separately



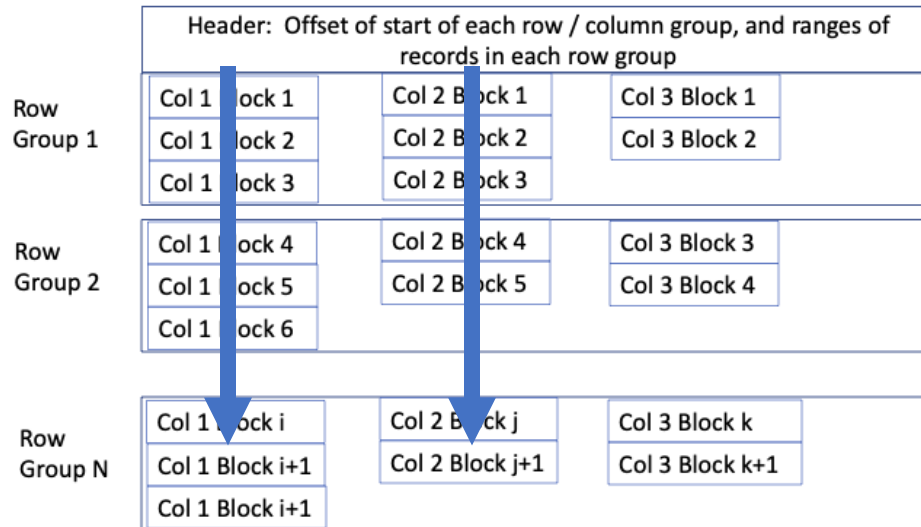
Using header, can efficiently read any subset of columns or rows without scanning whole file (unlike CSV)

Within a row group, data for each column is stored together

Predicate Pushdown w/ Parquet & Pandas

```
pd.read_parquet('file.pq', columns=['Col 1', 'Col 2'])
```

- Only reads col1 and col2 from disk
- For a wide dataset saves a ton of I/O



Performance Measurement

- Compare reading CSV to parquet to just columns we need

```
t = time.perf_counter()
df = pd.read_csv("FARS2019NationalCSV/Person.CSV", encoding = "ISO-8859-1")
print(f"csv elapsed = {time.perf_counter() - t:.3} seconds")

t = time.perf_counter()
df = pd.read_parquet("2019.pq")
print(f"parquet elapsed = {time.perf_counter() - t:.3} seconds")

t = time.perf_counter()
df = pd.read_parquet("2019.pq", columns = ['STATE', 'ST_CASE', 'DRINKING', 'PER_TYP'])
print(f"parquet subset elapsed = {time.perf_counter() - t:.3} seconds")
```

csv elapsed = 1.18 seconds

parquet elapsed = 0.338 seconds

parquet subset elapsed = 0.025 seconds

47x speedup

When to Use Parquet?

- Will always be more efficient than CSV
- Converting from Parquet to CSV takes time, so only makes sense to do so if working repeatedly with a file
- Parquet requires a library to access/read it, whereas many tools can work with CSV
- Because CSV is text, it can have mixed types in columns, or other inconsistencies
 - May be useful sometimes, but also very annoying!
 - Parquet does not support mixed types in a column

Summary

- Column oriented databases are a different way to “linearize” data to disk than the row-oriented representation we have studied
- A good fit for “warehousing” workloads that mostly read many records of a few tables
- It’s not as simple as just storing columns separately:
 - “Late materialization” execution
 - Column-specific compression and direct execution on compressed data
 - Read/write optimized stores
- Ideas have found their way into many modern systems and libraries, e.g., Parquet